

Day05 Part A 学习文档 v1.0 : Tool Calling 基础模型 (Execution Engine)

本文是《从零实现 Agent Runtime》学习阶段的 Day05 Part A 正式学习文档。

Day04 已经完成 Runtime 的认知系统 : Runtime State、Context Builder、Context Window Management 和 Provider Adapter。Day04.5 又补齐了工业术语映射。Day05 开始进入 Runtime 的执行系统，回答一个新的问题：Runtime 已经知道如何组织上下文让 LLM 思考，那么它如何让 Agent 真正行动？

本节定位

Day05 的主题是 Tool Calling，也就是 Agent Runtime 的 Execution Engine。

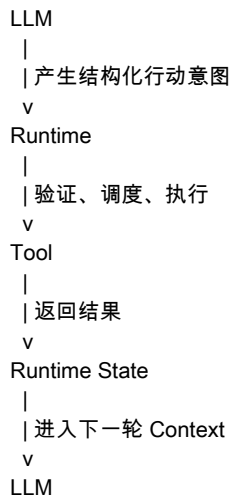
如果说 Day04 回答的是：

Runtime 如何思考？

那么 Day05 回答的是：

Runtime 如何行动？

Part A 不深入 Tool Schema、Tool Registry、Tool Executor 和 Permission 的细节，而是先建立 Tool Calling 的基础模型：



本节的核心结论是：

Tool Calling 不是 LLM 直接调用 API，而是 LLM 生成 Tool Call Intent，Runtime 负责验证、执行、观察结果并推动 Agent Loop 进入下一步。

目录

与上一节的联系

为什么需要 Tool Calling

ChatBot 与 Agent 的分水岭

Tool Calling 在 Runtime Loop 中的位置

Tool Calling 的三个角色

Tool / Function / Action 的区别

Function Calling 与 Tool Calling 的区别

ReAct 与 Tool Calling 的关系

工业框架中的 Tool Calling

mini-agent-runtime 中的最小数据模型

Part A 的边界

工业级实现 Notes

工业术语映射

面试视角

写书素材

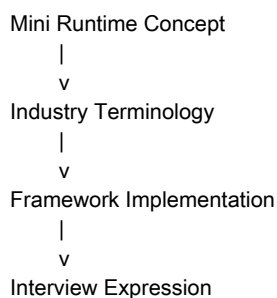
本 Part 核心认知升级

下一节学习计划

本章思考题

与上一节的联系

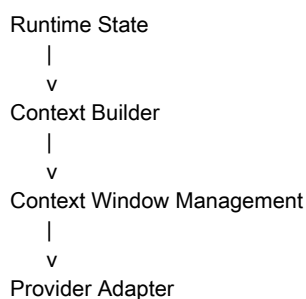
Day04.5 完成了一个 Terminology Mapping Pass :



这让我们之后可以直接使用工业术语：

- Runtime Loop / Agent Loop / ReAct Loop
- Runtime State / Agent State / Workflow State
- Context Builder / Context Engineering Pipeline
- Provider Adapter / LLM Gateway / Model Abstraction Layer
- Tool Runtime / Action Runtime / Tool Calling System

Day04 最终形成的是认知链路：



|
v
LLM

但到这里，Agent 仍然只能“想”和“说”。它还不能真正改变外部世界。

Day05 在这条链路之后补上执行链路：

LLM
|
v
Tool Call Intent
|
v
Runtime Tool Runtime
|
v
Environment / API / Filesystem / Browser / Database
|
v
Observation
|
v
Runtime State

所以 Day05 的位置可以概括为：

Day04:
Thinking System

Day05:
Execution System

为什么需要 Tool Calling

Day01 中我们已经给过 Agent 的基础定义：

Agent = LLM + Runtime + Tools + Memory

拆开来看：

- LLM 负责推理与生成；
- Runtime 负责控制与调度；
- Tools 负责让 Agent 影响外部世界；
- Memory 负责跨轮次、跨任务保留信息。

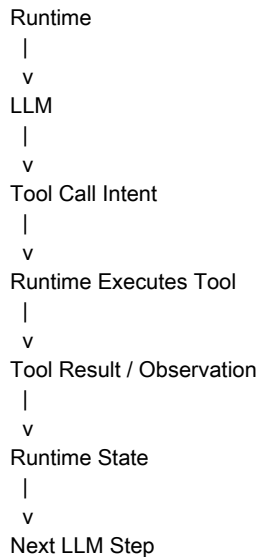
如果没有 Tool，系统链路只有：

User
|
v
LLM
|
v
Text Response

它可以生成文本、解释问题、总结资料、写代码片段，但它不能主动查询数据库、调用订单系统、读取文件、运行测试、打开浏览器或提交代码。

有了 Tool Calling 之后，链路变成：

User
|
v

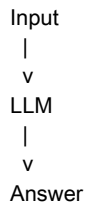


这时 LLM 不再只是回答问题，而是可以通过 Runtime 触发真实动作。

关键在于：LLM 仍然没有直接执行外部动作。它只是表达“我想调用哪个工具、带什么参数”。Runtime 才是真正的执行者。

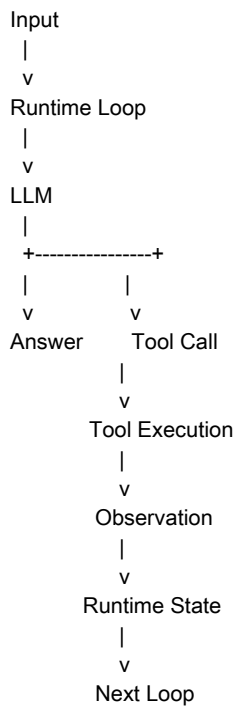
ChatBot 与 Agent 的分水岭

ChatBot 的典型模式是：



它的输出主要是自然语言。

Agent 的典型模式是：



差异不在于模型是否更聪明，而在于系统是否允许模型的推理结果进入行动闭环。

ChatBot 的能力边界是：

Talk

Agent 的能力边界是：

Think
|
v
Act
|
v
Observe
|
v
Continue

所以 Tool Calling 是 ChatBot 与 Agent 的分水岭。

Tool Calling 在 Runtime Loop 中的位置

Tool Calling 不是一个孤立 API，而是 Runtime Loop 中的一个阶段。

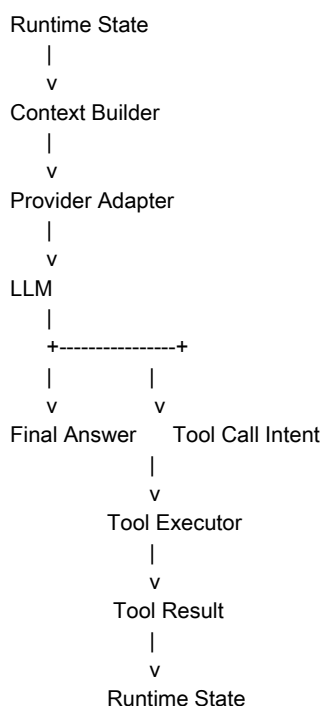
一个最小 Agent Loop 可以表示为：

```
while not done:
    context = context_builder.build(runtime_state)
    model_output = provider.generate(context, tools)

    if model_output contains final_answer:
        return final_answer

    if model_output contains tool_call:
        tool_result = tool_executor.execute(tool_call)
        runtime_state.add_observation(tool_result)
        continue
```

对应到结构图：



这里有两个非常重要的边界：

LLM 负责产生 Tool Call Intent；

Runtime 负责执行 Tool Call 并写回结果。

这也是 Day05 后续所有章节的基础。

Tool Calling 的三个角色

Tool Calling 至少包含三个角色。

1. Tool Definition

Tool Definition 是 Runtime 暴露给 LLM 的能力说明。

它通常包含：

- tool name；
- description；
- input schema；
- parameter constraints；
- permission requirement；
- execution metadata。

示例：

```
const searchTool = {  
  name: "search_files",  
  description: "Search files in the current workspace by keyword.",  
  inputSchema: {  
    type: "object",  
    properties: {  
      query: { type: "string" },  
      path: { type: "string" }  
    },  
    required: ["query"]  
  }  
};
```

Tool Definition 的本质不是“给函数写说明”，而是：

Runtime 向 LLM 暴露能力边界。

LLM 能否正确选择工具，很大程度取决于 Tool Definition 是否清晰、准确、边界明确。

2. Tool Decision

Tool Decision 是 LLM 根据当前 Context 和 Tool Definition 做出的选择。

它回答：

现在是否需要调用工具？
如果需要，调用哪个工具？
传入什么参数？

但这个阶段仍然只是决策，不是执行。

LLM 输出的不是 API 调用结果，而是结构化意图：

```
{
```

```
"tool_name": "search_files",
"arguments": {
  "query": "Runtime State",
  "path": "notes/"
}
}
```

3. Tool Execution

Tool Execution 属于 Runtime。

Runtime 会负责：

- 校验 tool name 是否存在；
- 校验 arguments 是否符合 schema；
- 检查 permission / guardrail；
- 执行真实函数或外部 API；
- 捕获异常和超时；
- 生成 Tool Result；
- 将结果写回 Runtime State。

这一步不能交给 LLM。

原因很简单：

LLM 可以提出行动意图，但不能被允许直接拥有副作用执行权。

Tool / Function / Action 的区别

这三个词经常混用，但在 Runtime 设计里最好区分。

Tool 是 Runtime 暴露给 Agent 的能力单元。

Tool = name + description + schema + executor + policy

Function 是某个编程语言里的可调用代码。

Function = code-level callable

Action 是 Agent Loop 中一次真实发生的动作。

Action = one executed step in the environment

三者关系：

```
Tool Definition
|
v
LLM chooses tool
|
v
Tool Call Intent
|
v
Runtime invokes Function
|
v
Action happens
|
v
Observation returns
```

也就是说：

- Tool 是能力抽象；
- Function 是代码实现；
- Action 是运行时行为。

Function Calling 与 Tool Calling 的区别

Function Calling 更偏模型输出格式。

它关注：

模型如何输出一个函数名和参数？

Tool Calling 是更大的 Runtime 概念。

它关注：

Runtime 如何向模型暴露能力、接收调用意图、验证参数、安全执行、记录结果、推动下一轮循环？

对比：

| 维度 | Function Calling | Tool Calling | | 关注点 | 模型输出结构 | Runtime 执行闭环 | | 范围 | 函数名 + 参数 | 定义、选择、执行、观察、状态回流 | | 执行者 | 外部代码自行处理 | Runtime Tool Executor | | 工业含义 | Provider API feature | Agent Runtime capability | | 结果语义 | function result | Observation / Tool Result |

所以：

Function Calling 是 Tool Calling 的一种接口形式；Tool Calling 是 Agent Runtime 的执行能力系统。

ReAct 与 Tool Calling 的关系

ReAct 是一种 Agent 推理模式：

```
Reason
|
v
Act
|
v
Observe
|
v
Reason Again
```

经典 ReAct 表达通常是：

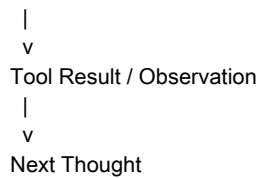
Thought:
I need to search for the order status.

Action:
query_order(order_id="123")

Observation:
The order has shipped.

Tool Calling 是 ReAct 中 Action 的结构化工程实现：

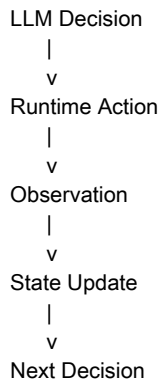
```
Thought
|
v
Tool Call
```

两者关系可以概括为：

ReAct 是认知模式
Tool Calling 是工程机制
Tool Result 是 Observation
Runtime Loop 是 ReAct Loop 的执行容器

因此，在 Agent Runtime 中不要把 ReAct 理解成一段 Prompt 技巧，而要理解成一种 Loop 结构：

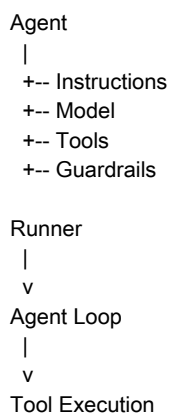


工业框架中的 Tool Calling

OpenAI Agents SDK

在 OpenAI Agents SDK 的语境里，Tool 不是一个普通函数列表，而是 Agent 可用能力的一部分。

可以从 Runtime 视角理解为：



这里的 Runner 更接近 Runtime Loop / Agent Orchestrator，Tool 被 Runner 调度执行，结果再回到后续模型调用中。

Claude Code

Claude Code 这类 Coding Agent 的强大之处，不只是模型会写代码，而是 Runtime 提供了一组真实工具能力：

Read File
Search
Edit File
Run Command

Inspect Git Diff
Run Tests
Ask Approval

从 Runtime 视角看：

```
graph TD
    A[LLM proposes edit] --> B[Runtime applies patch]
    B --> C[Filesystem changes]
    C --> D[Runtime observes diff/test result]
    D --> E[LLM continues]
```

Claude Code 不是“聊天模型加提示词”，而是 LLM 嵌入到一个受控的开发 Runtime 中。

LangGraph

LangGraph 更明确地把 Agent 看成状态图：

```
graph TD
    A[Node] --> B[State]
    B --> C[Edge]
    C --> D[Next Node]
```

Tool Calling 在 LangGraph 中可以理解为某个节点或边触发的 Action。Tool Result 更新 Graph State，之后路由到下一步。

对应关系：

Runtime State <-> Graph State
Tool Call <-> Tool Node / Action Step
Tool Result <-> State Update
Agent Loop <-> Graph Execution

mini-agent-runtime 中的最小数据模型

Part A 只需要建立最小模型，不提前实现完整 Tool Runtime。

Tool Definition

```
export interface ToolDefinition {
  name: string;
  description: string;
  inputSchema: JsonSchema;
}
```

Tool Call Intent

```
export interface ToolCallIntent {
  id: string;
  toolName: string;
  arguments: unknown;
```

```
}
```

Tool Result

```
export interface ToolResult {  
  toolCallId: string;  
  toolName: string;  
  status: "success" | "error";  
  output?: unknown;  
  error?: string;  
}
```

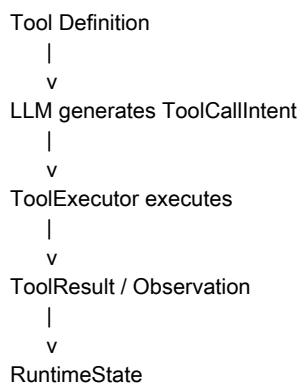
Tool Executor

```
export interface ToolExecutor {  
  execute(call: ToolCallIntent): Promise<ToolResult>;  
}
```

Runtime State 中的位置

```
export interface RuntimeState {  
  conversation: RuntimeMessage[];  
  pendingToolCalls: ToolCallIntent[];  
  observations: ToolResult[];  
}
```

这组模型表达的核心关系是：



Part C 会继续深入 Tool Schema，Part D 会设计 Tool Registry，Part E 会实现 Tool Executor。

Part A 的边界

学习过程中有一个节奏校准：Part A 一开始看起来比较短，但这是合理的。

Day05 已经拆成多个 Part：

- Part A：Tool Calling 基础模型
- Part B：LLM 如何决定调用 Tool
- Part C：Tool Schema 设计
- Part D：Tool Registry
- Part E：Tool Executor
- Part F：Permission & Human Approval
- Part G：Tool Result 回流 Runtime
- Part H：Multi Tool Loop
- Part I：Mini Tool Runtime 实现

如果 Part A 把 Tool Schema、Registry、Executor、Permission、Multi Tool Loop 都展开，会导致章节粒度失控。

因此 Part A 的结束标准不是内容多少，而是四个问题是否闭环：

核心认知是否建立；

是否和 Runtime 架构连接起来；

工业术语是否对齐；

后续章节依赖的问题是否已经铺垫。

Part A 已经完成：

- Tool Calling 为什么是 Agent 分水岭；
- Tool Calling 在 Runtime Loop 中的位置；
- Function Calling 与 Tool Calling 的区别；
- ReAct 与 Tool Calling 的关系；
- OpenAI Agents SDK、Claude Code、LangGraph 中 Tool 的定位；
- mini-agent-runtime 的最小数据模型。

因此 Day05 Part A 正式结束，下一节进入 Part B。

工业级实现 Notes

1. Tool Calling 不是 Provider Feature，而是 Runtime Feature

OpenAI、Claude 或 Gemini 可以提供 Tool Calling 的模型输出格式，但 Agent Runtime 不能把 Tool Calling 理解成某个 Provider 的 API 参数。

工业 Runtime 需要自己定义：

```
ToolDefinition
ToolCallIntent
ToolResult
ToolExecutionEvent
ToolPolicy
```

Provider Adapter 只负责转换格式。

2. Tool Execution 必须有控制边界

Tool 可能产生副作用，例如：

- 写文件；
- 执行命令；
- 调用支付 API；
- 修改数据库；
- 发送消息；
- 部署服务。

因此 Runtime 必须有：

- schema validation；
- permission check；
- timeout；
- retry policy；
- error mapping；

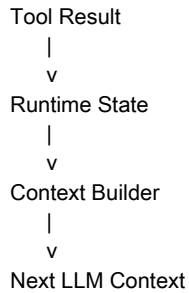
- audit log ;
- human approval。

这些内容会在 Day05 后续 Part 展开。

3. Tool Result 必须回流 Runtime State

Tool Result 不是简单返回给用户的文本。

它必须作为 Observation 写入 Runtime State :



否则 LLM 下一轮无法知道刚才发生了什么。

4. Agent 能力来自 Runtime 与 Tool 的组合

模型决定推理能力的上限，但 Runtime 决定这些推理能否变成稳定、可控、可观测的行动。

这也是为什么同一个模型放在普通 Chat 页面和 Claude Code 这类 Coding Agent 中，表现会完全不同。

工业术语映射

| 本课程概念 | 工业术语 | 说明 | |---| | Tool | Tool / Capability / Skill | Runtime 暴露给 Agent 的能力单元 | | Function | Function / Callable | Tool 背后的代码实现 | | Action | Action / Step | Agent Loop 中真实发生的一次动作 | | Tool Definition | Tool Schema / Tool Spec | 模型可见的工具契约 | | Tool Call Intent | Tool Call / Function Call | LLM 产生的结构化行动意图 | | Tool Executor | Tool Runtime / Action Executor | Runtime 中负责执行工具的组件 | | Tool Result | Observation | 工具执行后回到 Runtime 的结果 | | Tool Registry | Capability Registry | Runtime 管理可用工具的注册表 | | Permission | Guardrail / Policy | 执行前的安全和权限约束 | | Runtime Loop | Agent Loop / ReAct Loop | 思考、行动、观察、继续的执行循环 |

面试视角

Q1：为什么 Tool Calling 是 Agent 和 ChatBot 的区别？

ChatBot 主要把输入转换成文本输出。Agent 则通过 Tool Calling 把模型推理转换成真实环境中的动作，并把动作结果作为 Observation 回写到 Runtime State，进入下一轮决策。

Q2：Function Calling 和 Tool Calling 有什么区别？

Function Calling 更偏模型输出函数名和参数的能力；Tool Calling 是 Runtime 层面的执行系统，包含 Tool Definition、模型决策、参数验证、权限控制、真实执行、结果回流和下一轮 Agent Loop。

Q3：LLM 为什么不能直接调用 API？

因为 API 调用通常有副作用，需要权限、安全、审计、错误处理、重试和超时控制。LLM 只能产生结构化调用意图，Runtime 才能在受控边界内执行工具。

Q4 : Tool Result 为什么要写回 Runtime State ?

因为 Tool Result 在 ReAct 中就是 Observation。只有写回 Runtime State , Context Builder 才能在下一轮把观察结果投影给 LLM , Agent 才能继续推理。

写书素材

可以把 Tool Calling 解释成 :

Tool Calling is the moment an LLM-based system stops being only a language generator and starts becoming an actor in an environment.

但工程上要再补一句 :

The LLM does not execute the tool. It emits a structured intent. The Runtime validates, authorizes, executes, observes, and feeds the result back into state.

中文表达 :

Tool Calling 不是“模型会调函数”，而是 Runtime 给模型一套可描述、可验证、可执行、可回流的行动协议。

本 Part 核心认知升级

1. Tool Calling 不是调用函数

以前可能会理解成 :

LLM 调用函数

现在应该理解成 :

LLM 生成行动意图
Runtime 执行行动
Observation 回到 State

2. LLM 不执行 Tool

LLM 只是生成 :

tool_name + arguments

真正执行的是 :

Runtime Tool Executor

3. Tool Calling 是 State Transition

Tool Calling 不只是一次外部调用，而是推动 Runtime State 演化的一次 Action :

```
State_before
|
v
Tool Action
|
v
Observation
|
v
State_after
```

4. Tool Schema Engineering 会影响 Agent Behavior

Day04 学的是：

Runtime 决定给 LLM 看什么

Day05 会进一步学习：

Runtime 如何影响 LLM 决定做什么

二者合在一起就是：

Context Engineering
+
Tool Schema Engineering
|
v
Agent Behavior Engineering

下一节学习计划

Day05 Part B：LLM 如何决定调用 Tool。

下一节重点回答：

LLM 如何知道当前需要 Tool；

Tool Schema 如何影响模型决策；

Tool Choice 中 auto、required、none 的区别；

Tool Calling 的决策链；

ReAct 与 Function Calling 在决策层的关系；

OpenAI、Claude、LangGraph 的实现差异。

本章思考题

为什么说 Tool Calling 不是 LLM 直接调用 API？

Tool、Function、Action 三者有什么区别？

Tool Call Intent 为什么必须由 Runtime 验证后再执行？

Tool Result 为什么在 ReAct 语境下等价于 Observation？

如果 Tool Result 不写回 Runtime State，Agent Loop 会出现什么问题？

为什么 Claude Code 这类 Coding Agent 的能力不只来自模型？

Source

- ChatGPT 分享学习记录：<https://chatgpt.com/share/6a62c9e6-e190-83ee-9020-4fb6785886fe>

- 本地源记录：source/day05-part-a-chatgpt-share-source.md